

S6. Redis Streams 내부 구조와 At-least-once 처리 보장 원리

Pub/Sub vs Streams · Consumer Group · PEL · XACK · 재처리 메커니즘

2026.05 | M2 S6

S5 복습 — 우리가 만든 것

S5 완료: WebhookReceiver + QueueService(stub) + 202 패턴

VASP
이벤트 전송

WebhookReceiver
POST /webhook
202 즉시 반환

QueueService
enqueue()
← stub 상태

Redis Streams
??? 실제 구현
← 오늘 주제

오늘 목표: QueueService stub → Redis Streams 실제 구현으로 교체

Redis Streams를 이해해야 enqueue() / XREADGROUP / XACK 구현 가능

S6 — 오늘 배울 핵심 4가지

1

Pub/Sub · Queue · Streams
3가지 차이와 선택 이유

2

Redis Streams 구조
XADD · Entry · Consumer Group

3

PEL + XACK
At-least-once 보장 원리

4

Consumer Group
수평 확장 메커니즘

완료 기준: PEL + XACK 장애 재처리 경로 설명 / Consumer Group 분산 원리 설명

Redis 간단 복습 — 무엇인가

Redis = Remote Dictionary Server = 인메모리 데이터 구조 저장소

메모리 기반

디스크보다 100~1000배 빠름
처리 속도 극대화

영속성 옵션

RDB(스냅샷) / AOF(로그)
재시작 후에도 데이터 보존

다양한 자료구조

String / List / Hash
Set / Sorted Set / Streams

단일 스레드

명령 처리 순서 보장
동시성 문제 없음

교보 프로젝트: Redis Streams 자료구조 사용 → 이벤트 큐 + Consumer Group 관리

왜 Redis Streams인가 — 결론 먼저



영구 저장

서버 재시작 후에도
이벤트 손실 없음



Consumer Group

여러 인스턴스 병렬 처리
중복 없는 분배



PEL

처리 완료(XACK) 전까지
메시지 안전하게 보관



재처리

XACK 전 장애 시
자동으로 재수신 가능



운영 단순성

별도 브로커 불필요
Redis 하나로 Queue 구현

S6 전체: 위 5가지가 어떻게 동작하는지 상세히 이해하는 시간

교보 Phase 1 이벤트 파이프라인 = Redis Streams 선택. 이유를 이해해야 활용 가능.

메시지 전달 3가지 패턴 — 한눈에 비교

Pub/Sub		단순 Queue	Redis Streams
저장 방식	메모리 (일시적)	메모리 (일시적)	메모리+AOF (영구)
구독자 없을 때	메시지 유실	큐에 보관	스트림에 영구 보관
재수신 가능	✗ 불가	△ 구현에 따라	✓ 언제든지 가능
Consumer Group	✗ 없음	△ 제한적	✓ 내장 지원
처리 확인(ACK)	✗ 없음	△ 구현에 따라	✓ XACK
메시지 순서	보장 어려움	FIFO 보장	FIFO + ID 기반

Redis Streams = 세 가지 중 가장 강력한 내구성 + 재처리 보장

Pub/Sub — 왜 이벤트 파이프라인에 부족한가

Pub/Sub 구조 — 발행자와 구독자 실시간 연결

Publisher
VASP



Redis
Pub/Sub Channel



Subscriber
Consumer

구독자 없을 때 발행

메시지 즉시 사라짐
→ 이벤트 영구 유실

구독자 장애 시

재시작 후 복구 불가
→ 장애 간 이벤트 모두 유실

구독자 추가 시

기존 메시지 수신 불가
→ 과거 이벤트 재처리 불가

결론: Pub/Sub은 실시간 알림에 적합. 내구성 필요한 이벤트 파이프라인에는 부적합.

단순 Queue — 무엇이 부족한가

단순 Queue — 메시지 저장 + FIFO 처리

Producer
VASP



Simple Queue
(Redis List 등)



Consumer
처리

처리 확인 없음

메시지 꺼내는 순간 삭제
처리 실패해도 복구 불가

Consumer 확장 어려움

여러 인스턴스 동시 소비 시
중복 처리 발생 가능

재처리 불가

처리된 메시지 재수신 불가
오류 분석·재처리 어려움

결론: 단순 Queue는 기본 내구성 제공. ACK + Consumer Group이 없어서 금융 파이프라인에 부족.

Redis Streams — 무엇이 다른가

Redis Streams = 영구 로그 + Consumer Group + ACK 통합

영구 로그 구조

XADD로 추가만 됨
삭제 명시 전까지 보존
과거 재조회 가능

Consumer Group

그룹 단위 소비
그룹 내 각 메시지 1회만 전달
여러 Consumer 안전 분배

PEL

Pending Entry List
읽혔지만 XACK 안 된 목록
장애 시 재수신 기준

XACK

처리 완료 선언
PEL에서 제거
XACK 없으면 영원히 PEL에
잔류

이 4가지가 조합되어 At-least-once 처리 보장이 가능해진다

선택 기준 — 언제 무엇을 쓰는가

상황별 선택 가이드

실시간 알림

채팅, 실시간 이벤트
구독자 없으면 유실 허용

Pub/Sub ✓

단순 작업 큐

이미지 처리 요청
빠른 구현, 내구성 불필요

Simple Queue ✓

금융 이벤트 파이프라인

NFT 발행, 원장 업데이트
내구성 + 재처리 + ACK 필수

Redis Streams ✓

초대형 스트리밍

초당 수백만 이벤트
분산 클러스터 필요

Kafka ✓

교보 Phase 1 = 금융 이벤트 파이프라인 → Redis Streams

Kafka vs Redis Streams — 교보 프로젝트 선택 이유

Apache Kafka		Redis Streams
처리량	초당 수백만 건	초당 수십만 건
운영 복잡도	ZooKeeper/KRaft 필요	Redis 하나로 끝
학습 곡선	높음 (별도 개념체계)	낮음 (Redis 명령어)
지연 시간	ms 단위	μs 단위 가능
교보 Phase 1	오버스펙	적합

Phase 2+ 확장 시 Kafka 이전 고려 가능. Phase 1은 Redis Streams로 충분.

메시지 처리 보장 수준 — At-most-once vs Exactly-once vs At-least-once

	At-most-once	Exactly-once	At-least-once
정의	최대 1번 처리 (0번일 수도 있음)	정확히 1번 처리	최소 1번 처리 (2번일 수도 있음)
특성	손실 가능 중복 없음	손실 없음 중복 없음	손실 없음 중복 가능
적합 상황	간단한 알림 손실 허용 시	분산 환경에서 사실상 불가	금융 시스템 멥등성으로 보완
교보 선택	✗ 금융 부적합	⚠ 구현 불가	✓ 교보 채택

At-least-once + 멍등성 = 안전한 분산 처리

At-least-once 보장

- 처리 실패 시 재처리 가능
- 메시지 절대 유실 없음
- PEL에 잔류 → 재수신
- XACK 전까지 안전 보관

멍등성 (Idempotency)

- 동일 처리 N번 = 결과 동일
- txHash 기준 중복 체크
- ON CONFLICT DO NOTHING
- 재처리 안전 보장

At-least-once + 멍등성 = 금융 시스템의 현실적 최선
Exactly-once 불가 → 재처리 허용 + 중복 방지로 동일 결과 보장

Redis Streams = 추가 전용 로그 자료구조

Streams = 순서가 있는 항목들의 영구 시퀀스

Entry 1: 1700000000001-0 | eventType=NFT_ISSUED | txHash=0xabc1... | tokenId=1

Entry 2: 1700000000002-0 | eventType=NFT_ISSUED | txHash=0xabc2... | tokenId=2

Entry 3: 1700000000003-0 | eventType=NFT_ISSUED | txHash=0xabc3... | tokenId=3

Entry 4: 1700000000004-0 | eventType=NFT_ISSUED | txHash=0xabc4... | tokenId=4

Entry 5: 1700000000005-0 | eventType=NFT_ISSUED | txHash=0xabc5... | tokenId=5



XADD = 항목 추가 (삭제 없음). ID는 timestamp-seq 형식으로 자동 생성. 순서 보장.

Stream Entry 구조 — ID + Field-Value 쌍

모든 Stream Entry는 고유 ID와 field-value 쌍으로 구성

Entry ID: 1700000001234-0

timestamp(ms)-seq

- timestamp: 밀리초 단위 Unix 타임스탬프
- seq: 동일 ms 내 순서 번호 (0부터)
- Redis가 자동 생성 — 충돌 없음 보장
- 정렬 기준 = 시간 순 보장

Field-Value 쌍 (해시 구조)

eventType=NFT_ISSUED

- txHash=0xabc123...
- tokenId=42
- recipient=0xdef...
- blockNumber=19000001

XADD — 이벤트 스트림에 항목 추가

XADD key ID field value [field value ...]

XADD

명령어

eventStream

스트림 이름 (키)

*

ID 자동 생성
(직접 지정도 가능)

eventType
NFT_ISSUED

field=value 쌍
여러 개 가능

반환값: "1700000001234-0" (생성된 Entry ID)

MAXLEN ~1000

스트림 크기 제한 (근사치)
오래된 항목 자동 삭제

NOMKSTREAM

스트림 없으면 생성 안 함
(기본: 자동 생성)

스트림 조회 — XLEN / XRANGE

스트림 상태 확인 명령어

XLEN eventStream

스트림 항목 수 반환
→ 정수값

XRANGE eventStream - +

전체 항목 조회
- = 처음 / + = 끝

XRANGE eventStream - + COUNT 10

최대 10개만 조회

XREVRANGE eventStream + -

역순 조회
(최신→오래된 순)

XREAD COUNT 10 STREAMS eventStream 0

특정 ID 이후 읽기
0 = 처음부터

S7 CLI 실습에서 직접 타이핑해볼 명령어들 — 오늘은 구조만 이해

Consumer Group — 왜 필요한가

Consumer Group = 여러 Consumer가 하나의 스트림을 분담하는 구조

Consumer Group 없이 — 문제 상황

- Consumer A와 B가 같은 스트림을 읽으면
- 동일 메시지를 둘 다 읽게 됨
- 중복 처리 발생
- 처리 완료 추적 불가

Consumer Group 있을 때 — 해결

- Redis가 메시지를 Consumer에 1:1 배분
- 동일 메시지 = 하나의 Consumer만
- 중복 처리 없음
- 처리 상태 Redis가 추적

Consumer Group = 분산 처리의 핵심 메커니즘

Consumer Group 생성 — XGROUP CREATE

XGROUP CREATE eventStream nftConsumers \$ MKSTREAM

XGROUP CREATE

Consumer Group
생성 명령어

eventStream

대상 스트림 이름

nftConsumers

그룹 이름
(임의 지정)

\$

지금 이후 메시지만
(0이면 처음부터)

MKSTREAM

스트림 없으면
자동 생성

⚠ \$ vs 0: \$ = 지금 이후 새 메시지만. 0 = 처음부터 모든 메시지. 잘못 선택 시 과거 이벤트 누락

교보 프로젝트: QueueService 초기화 시 XGROUP CREATE 실행 — 이미 존재하면 에러 무시(BUSYGROUP)

여러 Consumer Group — 독립적 소비

같은 스트림에 여러 Consumer Group = 각 Group이 독립적으로 전체 스트림 소비

Group A: nftConsumers

Consumer A1

Entry 1,3,5
→ A1

Consumer A2

Entry 2,4
→ A2

eventStream

Entry 1
Entry 2
Entry 3
Entry 4
Entry 5

Group B: auditConsumers

Entry 1,2,3,4,5
전부 B1

Consumer B1

Group A와 Group B는 같은 스트림을 독립적으로 소비 — 한쪽 처리가 다른 쪽 영향 없음

교보: nftConsumers(NFT 처리) + auditConsumers(감사 로그) 별도 그룹 가능

Consumer 등록과 식별

Consumer = Consumer Group 내의 개별 처리 인스턴스

```
XREADGROUP GROUP nftConsumers consumer-1 COUNT 10 STREAMS eventStream >
```

nftConsumers

Consumer Group 이름

consumer-1

이 Consumer의 식별자 (자동 생성됨)

>

미처리 새 메시지만 읽기 (PEL 제외)

COUNT 10

최대 10개 한 번에

consumer-1 이름은 Redis가 자동 등록 — 처음 XREADGROUP 호출 시 생성됨

PEL — Pending Entry List

PEL = Pending Entry List
읽혔지만 아직 XACK되지 않은 메시지 목록

Stream (전체 이벤트)

Entry 1  XACK됨
Entry 2  PEL에 있음
Entry 3  PEL에 있음
Entry 4  아직 미읽음

PEL (처리 중 목록)

Entry 2 → consumer-1
delivered_at: ...
delivery_count: 1
Entry 3 → consumer-2
delivery_count: 1

PEL = "처리 중" 표시판. XACK 받아야 제거됨. 장애 시 재처리 기준.

XREADGROUP — > 심볼의 의미

XREADGROUP에서 ID 위치에 > 를 쓰면 — 미처리 새 메시지만

>

새 메시지만 읽기
PEL에 없는, 아직 어떤 Consumer도 안 읽은 메시지

정상 운영 시 사용

0

PEL에 있는 메시지
읽었지만 XACK 안 된 메시지 재수신

장애 후 재시작 시 사용

1234-0

특정 ID 이후 PEL 메시지
해당 ID보다 큰 PEL 항목

세밀한 재처리 제어

핵심: 장애 후 재시작 시 > 대신 0 사용 → PEL 메시지 재수신

XREADGROUP 처리 흐름



XREADGROUP 호출 → Redis가 미할당 Entry 반환 + 동시에 PEL에 등록

반환된 Entry는 Consumer가 처리 중인 상태로 표시됨 — 다른 Consumer에게 재배포 안 됨

XACK — 처리 완료 선언

XACK streamKey groupName messageId

XACK eventStream nftConsumers 1700000001234-0

PEL에서 제거

해당 Entry가 Pending 목록에서 사라짐

처리 완료 기록

Redis가 해당 Entry를 처리됨으로 표시

메시지 보존

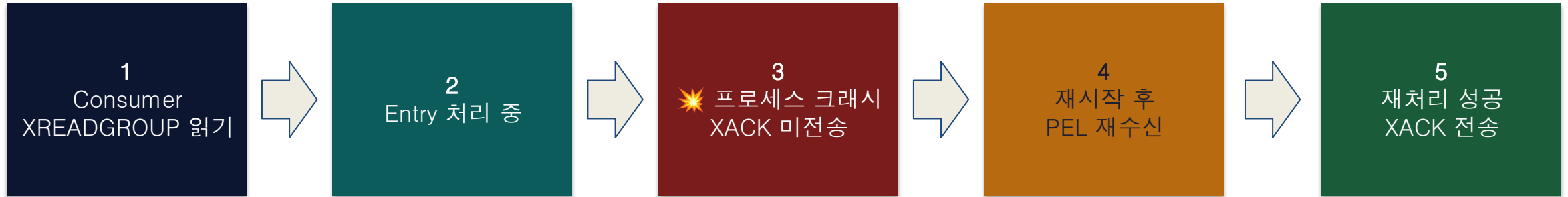
Stream에서는 삭제 안 됨 — 로그는 유지

재수신 없음

다른 Consumer가 이 Entry를 받지 않음

⚠ XACK를 보내지 않으면 — PEL에 영구 잔류 → 장애 후 재수신 → 중복 처리 위험

XACK 전 장애 — 무슨 일이 일어나는가



4단계: 재시작 시 XREADGROUP 0 호출 → PEL에 있는 미ACK 메시지 재수신

XACK를 보내기 전에 Consumer 프로세스가 죽었다면?

5단계에서 먹등성 체크 → 이미 처리됐으면 XACK만 전송 (원장 변경 없음)

XPENDING — PEL 상태 확인

XPENDING eventStream nftConsumers - + 10

응답 예시: 1700000001234-0 consumer-1 12000 3

Entry ID	1700000001234-0	메시지 식별자
Consumer Name	consumer-1	현재 처리 중인 Consumer
Idle Time (ms)	12000	마지막 전달 이후 경과 시간 (ms)
Delivery Count	3	전달 횟수 — 높으면 반복 실패 의심

Delivery Count ≥ 3 = 처리 반복 실패 → DLQ 이동 기준 (S11)

XAUTOCLAIM — 오래된 미ACK 메시지 재배포

XAUTOCLAIM eventStream nftConsumers consumer-2 60000 0-0

consumer-2

이 Consumer가 메시지를 인계받음

60000

60초 이상 미ACK인 메시지만 대상

0-0

이 ID부터 스캔 시작 (전체)

사용 시점: Consumer A가 죽고 PEL에 메시지가 쌓일 때
Consumer B가 XAUTOCLAIM으로 인계받아 처리

교보: Delivery Count 기반 DLQ 이동 로직에서 XAUTOCLAIM 활용 (S11 상세)

XAUTOCLAIM은 클러스터 내 자동 부하 재배포의 핵심 도구

PEL Entry 생명주기 — 탄생부터 소멸까지

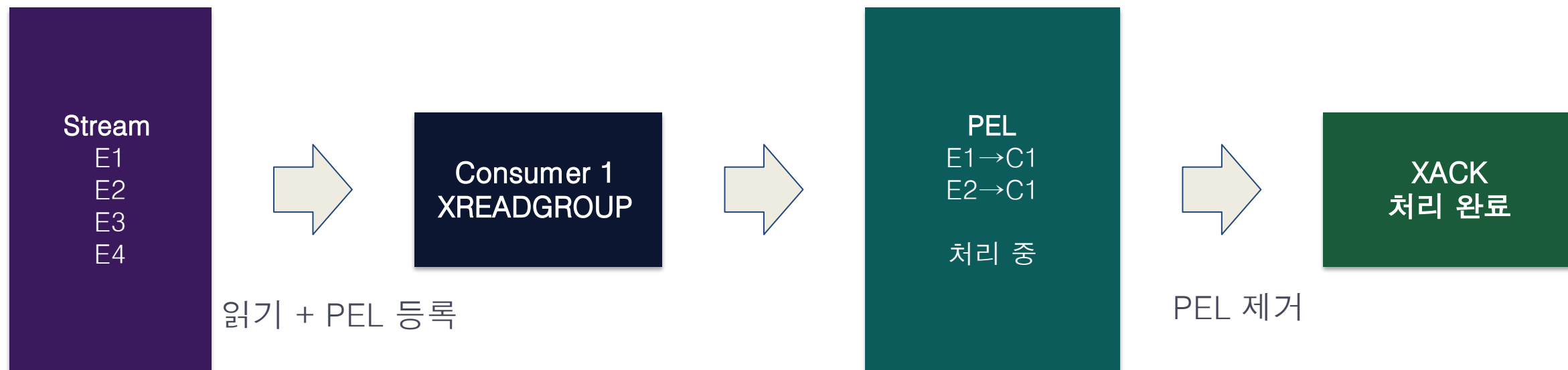


DLQ = Dead Letter Queue. 반복 실패 메시지 격리. 수동 검토 후 재큐잉.

정상 경로: 탄생 → 잔류 → 정상 종료
장애 경로: 탄생 → 잔류 → (장애) → 재처리 → 정상 종료 (또는 DLQ)

이 생명주기가 At-least-once를 구현하는 핵심 메커니즘

PEL + XACK 전체 흐름 — 정상 시나리오

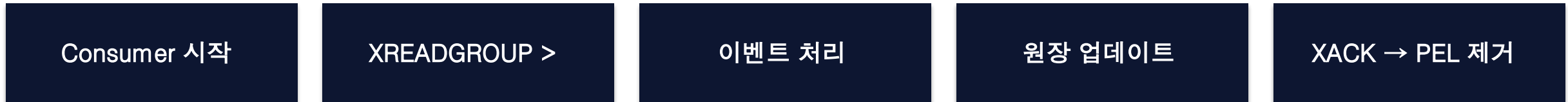


XREADGROUP → 처리 → XACK. 이 3단계가 At-least-once의 전부.

At-least-once 보장 — 전체 원리

At-least-once = PEL + XACK + 재시작 후 PEL 재수신의 조합

정상 경로



장애 경로



장애가 발생해도 PEL에 메시지가 살아있기 때문에 재처리 가능 = At-least-once

재시작 후 복구 — 무엇을 해야 하는가

Consumer 재시작 시 복구 순서

1단계	XREADGROUP 0 호출	PEL에 있는 미ACK 메시지 우선 재수신
2단계	역등성 체크	txHash로 이미 처리됐는지 확인
3단계A	미처리 메시지	정상 처리 → 원장 업데이트 → XACK
3단계B	이미 처리된 메시지	원장 변경 없음 → XACK만 전송
4단계	PEL 비면	XREADGROUP > 로 전환 → 새 메시지 처리

역등성 체크가 없으면 3단계B에서 중복 원장 업데이트 발생

XACK 실패 — 이 경우는 어떻게 되는가

상황: DB 커밋 성공 → XACK 호출 → 네트워크 장애 → XACK 실패

PEL에 잔류

XACK 없음 → PEL에 계속 존재

재시작 후

0으로 XREADGROUP → 같은 메시지 재수신

역등성 체크

txHash 이미 처리됨 → DB 건드리지 않음

XACK 재전송

역등성 확인 후 XACK만 전송 → 정리 완료

처리는 성공했는데 XACK 전송이 실패하면?

결론: XACK 실패도 안전. 역등성이 있으면 재처리 = 안전한 재ACK만 발생.

Redis-DB 원자성 불가 문제 — 현실적 해결책

Redis(XACK)와 PostgreSQL은 2PC(Two-Phase Commit) 불가 — 원자적 처리 불가능

시나리오 A

DB 커밋 성공
XACK 실패

→ 멍등성으로 해결
재처리 시 DB 건드리지 않음

시나리오 B

XACK 성공
DB 커밋 실패

→ PEL에서 사라짐
메시지 영구 유실 위험
→ DB 트랜잭션 먼저가 중요

올바른 순서

멍등성 체크 → DB 트랜잭션
→ 커밋 → XACK

→ 시나리오 B 방지
순서 절대 고정

S9에서 처리 순서(멍등성→DB→커밋→XACK)를 코드로 구현하는 이유

At-least-once 보장 요약

At-least-once 보장 = 3가지 메커니즘의 조합

PEL

읽혔지만 ACK 안 된
메시지 추적
장애 기준점

XACK

처리 완료 선언
PEL 제거
재수신 방지

멱등성

중복 처리 방지
txHash 체크
결과 동일 보장

At-least-once + 멱등성
= 금융 시스템 안전 처리

delivery_count — 반복 실패 감지와 DLQ

XPENDING의 delivery_count = 이 메시지가 몇 번 전달됐는가

delivery_count 1

처음 처리 시도
정상 경로

delivery_count 2

1번 실패 후
재처리 중

delivery_count 3

2번 실패
마지막 시도

delivery_count ≥ 3

처리 불가 판단
→ DLQ 이동

DLQ = Dead Letter Queue. 3회 실패 메시지를 별도 스트림으로 격리 후 수동 검토.

S11에서 moveToDLQ() 구현. XAUTOCLAIM으로 delivery_count 조회 후 판단.

스트림 크기 관리 — MAXLEN

무한정 쌓이는 스트림 — 어떻게 관리하는가

```
XADD eventStream MAXLEN ~ 10000 * eventType NFT_ISSUED ...
```

MAXLEN ~ 10000 = 근사치로 10000개 제한 (정확한 제한보다 빠름)

MAXLEN 설정 없음

무한 성장 → 메모리 부족 위험
장기 운영 시 Redis OOM

MAXLEN ~ N (근사치)

효율적 크기 제한
전략적 보존 기간 설정 가능

MAXLEN = N (정확치)

정확하지만 느림
대량 트래픽에서 성능 저하

교보: MAXLEN ~ 100000 (약 10만 건) 설정 권장. 처리 완료 후 자동 정리.

At-least-once 이해 완료 기준

이것을 설명할 수 있으면 S6 이해 완료

Q1

PEL이 무엇이고, 언제 생성되고, 언제 사라지는가?

A: XREADGROUP 읽기 시 생성 / XACK 시 제거

Q2

XACK 전 Consumer 장애 시 무슨 일이 일어나는가?

A: PEL 잔류 → 재시작 후 XREADGROUP 0 → 재수신

Q3

XACK 후 DB 커밋이 실패하면?

A: XACK 먼저 금지 → DB 커밋 먼저, XACK 나중

Q4

Consumer 2개가 같은 그룹에 있을 때 메시지 분배는?

A: Redis가 자동 분배 — 동일 메시지는 하나의 Consumer만

Consumer Group 수평 확장 — 인스턴스 추가 = 처리량 증가

Consumer 인스턴스 추가 = 처리량 선형 증가 (이론적)

Consumer 1개

E1

E2

E3

E4



Consumer 4개 (수평 확장)

C1: E1~3

C2: E4~6

C3: E7~9

C4: E10~12

처리량: 10 TPS

처리량: ~40 TPS (병렬 처리)

Consumer Group이 중복 없이 자동 분배 → 코드 변경 없이 인스턴스만 추가

Docker/K8s에서 replica 수만 늘리면 됨. QueueService 코드 변경 없음.

Consumer 추가 시 Redis 내부 동작

동작 원리: 각 Consumer가 XREADGROUP 호출 시 미할당 메시지 반환

Consumer 3 추가
(XREADGROUP 호출)

Redis: 미할당 E7, E8 있음
→ Consumer 3에게 반환

Consumer 1,2
계속 처리 중

E1~E6: 이미 할당됨
→ Consumer 3 배정 안 됨

E9, E10 새 이벤트
들어옴

Consumer 1~3 중
가장 먼저 XREADGROUP 부른 쪽

Consumer가 추가되면 Redis는 어떻게 메시지를 분배하는가?

부하 분산은 Redis가 알아서 — 우리 코드: Consumer 수만 제어

Consumer 장애/제거 시 동작

Consumer 하나가 갑자기 죽었다 — 무슨 일이 일어나는가?

이전 상태

Consumer 3개
E1~E3 각각 할당
처리 중

C2 크래시

E2: PEL에 잔류
(C2 처리 안 됨)
다른 Consumer 영향 없음

XAUTOCLAIM

60초 후 자동 감지
E2 → C1 또는 C3 재배포
delivery_count +1

정상화

C1/C3가 E2 처리
+ XACK → PEL 정리
전체 처리 계속

Consumer 하나가 죽어도 PEL + XAUTOCLAIM으로 자동 복구 — 운영 팀 개입 최소화

교보 프로젝트 Consumer 운영 패턴

Phase 1 MVP 기준 Consumer Group 구성

nftConsumers

NFT_ISSUED / NFT_BURNED
처리

Consumer × 2
(Primary + Backup)

auditConsumers

모든 이벤트
감사 로그 기록

Consumer × 1
(감사 전용)

dlqConsumers

DLQ 수동 재처리
운영팀 검토 후

수동 트리거
(자동 실행 없음)

Consumer 수 확장: docker-compose replicas 수정 또는 K8s HPA 설정

처리량 계산 — Consumer 수 어떻게 결정하는가

처리량 요구사항 → Consumer 수 산정

VASP 이벤트 수신

피크: 100 TPS
(NFT 발행 이벤트 기준)

Consumer 처리 시간

평균: 50ms/건
= 20 TPS/Consumer

필요 Consumer 수

$100 \text{ TPS} \div 20 \text{ TPS} = 5\text{개}$
+ 여유분 2개 = 7개

Phase 1 MVP

피크 예상: 10 TPS
→ Consumer 2개로 충분

부하 테스트로 실측 → Consumer 수 조정. 처음부터 과도하게 늘릴 필요 없음.

Consumer Group 운영 모니터링

XINFO GROUPS eventStream — 그룹 상태 확인

name	그룹 이름	nftConsumers
consumers	현재 Consumer 수	3
pending	PEL에 있는 메시지 수	0 (정상) / 높으면 경고
last-delivered-id	마지막 배달 ID	처리 진행 상황 확인
entries-read	처리된 총 메시지 수	누적 처리량
lag	미처리 메시지 수	0이 목표 / 지속 증가 시 Consumer 추가

lag > 0 지속 증가 = Consumer 처리 속도 < 이벤트 유입 속도 → Consumer 추가

수평 확장 한계 — 언제 Redis Streams가 부족한가

Redis Streams의 한계 — 이 시점이 오면 Kafka로 전환 검토

단일 노드 한계

Redis 단일 노드: 메모리·CPU 상한 존재
초당 수십만 건 이상 시 한계

파티셔닝 없음

Kafka와 달리 파티션 분산 없음
스트림 자체를 분산할 수 없음

클러스터 복잡성

Redis Cluster 구성 시 복잡도 증가
Slot 기반 샤딩 이해 필요

교보 Phase 1

현 규모에서 문제 없음
Phase 2+ 트래픽 급증 시 재검토

COINCRAFT RESEARCH

정리

오늘 배운 것 · 핵심 개념 요약 · 다음 세션 예고

S6 마무리

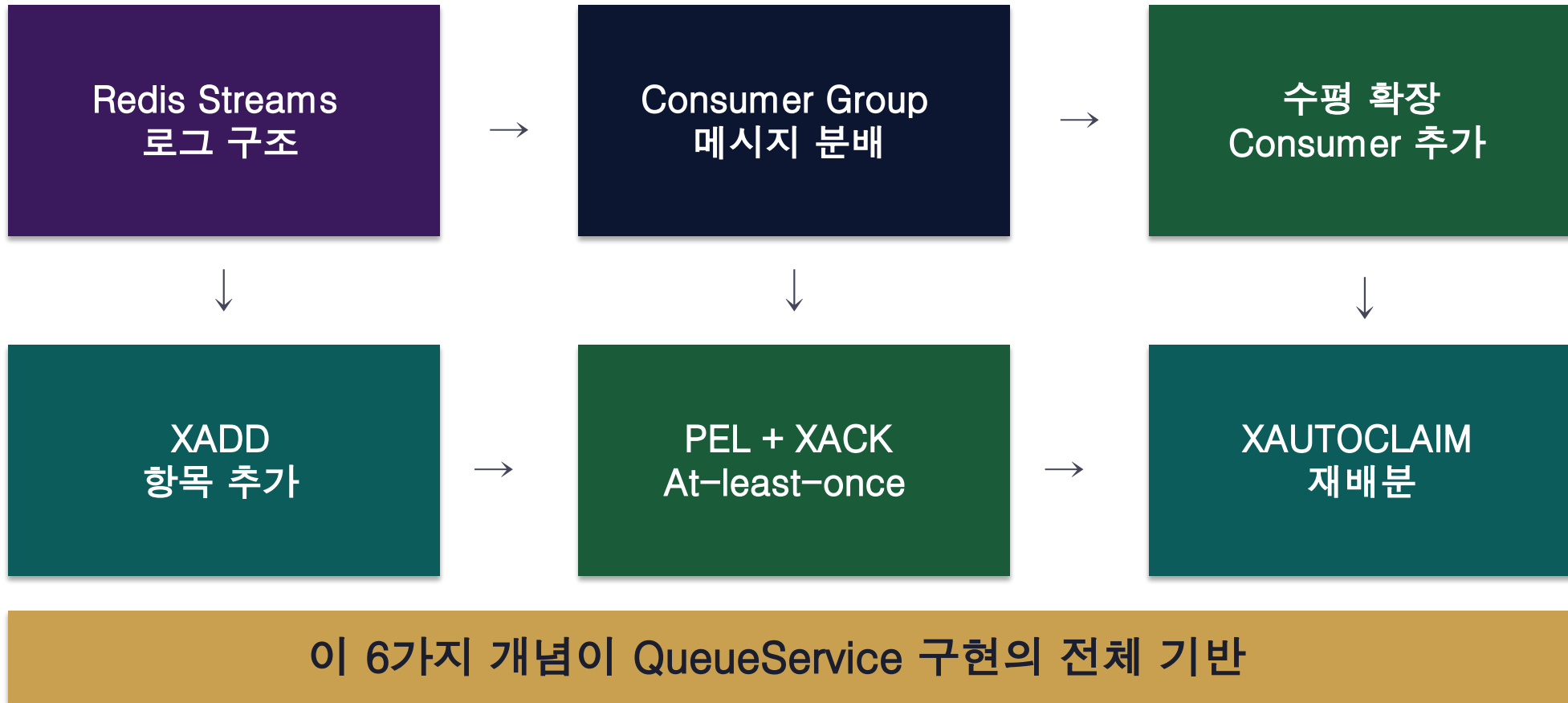
COINCRAFT
ACADEMY

S6 핵심 Redis 명령어 모음

오늘 배운 명령어 — S7 실습에서 직접 타이핑

XADD	스트림에 항목 추가	XADD stream * field val
XGROUP CREATE	Consumer Group 생성	XGROUP CREATE stream group \$ MKSTREAM
XREADGROUP	그룹 기반 읽기	XREADGROUP GROUP g c COUNT N STREAMS s >
XACK	처리 완료 선언	XACK stream group messageid
XPENDING	PEL 조회	XPENDING stream group - + N
XAUTOCLAIM	오랜 미ACK 재배포	XAUTOCLAIM stream group c timeout 0-0
XINFO GROUPS	그룹 상태 확인	XINFO GROUPS stream

S6 개념 지도 — 전체 연결



S6 완료 기준

이것을 설명할 수 있어야 S6 완료



PEL 설명

PEL이 무엇이고 언제 생성·제거되는지



XACK 이해

XACK 전 장애 시 재처리 경로



Pub/Sub vs Streams

왜 Pub/Sub이 아닌 Streams를 쓰는지



Consumer Group 분산

여러 Consumer가 중복 없이 처리하는 원리



At-least-once

멥등성과 결합하여 안전한 처리가 되는 이유

다음 세션 (S7): Redis Streams CLI 실습 + At-least-once 재처리 시뮬레이션

다음 세션 — S7 Redis Streams CLI 실습

S6 이론 → S7 실습: 직접 타이핑해서 원리 확인

Redis 기동

로컬 Redis 인스턴스 시작 확인

XADD 실습

이벤트 항목 직접 스트림에 적재

Consumer Group 생성

XGROUP CREATE 실행 + 초기화

XREADGROUP 읽기

메시지 읽기 + PEL 등록 확인

XACK 처리

ACK 전송 + PEL 제거 확인

미ACK 시뮬레이션

XACK 전 종료 → 재시작 → 재수신 확인

Consumer 2개 분배

2개 Consumer 동시 실행 → 메시지 분배 확인

S7 완료 = Redis Streams 손에 익히기 완료 → S8 QueueService 실제 구현

핵심 용어 정리 — S6

PEL	Pending Entry List. 읽혔지만 XACK 안 된 메시지 목록.
XADD	Stream에 항목 추가. ID 자동 생성.
XREADGROUP	Consumer Group 기반 메시지 읽기. > = 새 메시지, 0 = PEL 재수신.
XACK	처리 완료 선언. PEL에서 제거.
XPENDING	PEL 내용 조회. delivery_count 확인.
XAUTOCLAIM	오랫동안 미ACK 메시지를 다른 Consumer로 재배포.
Consumer Group	여러 Consumer가 하나의 스트림을 분담하는 구조.
At-least-once	최소 1번 처리 보장. PEL + XACK의 조합으로 구현.

수고하셨습니다

S6 — Redis Streams 내부 구조와 At-least-once 처리 보장 원리

오늘 완성한 이해

Streams 구조 ✓
PEL + XACK ✓
At-least-once ✓
Consumer Group ✓

다음 세션 (S7)

CLI 실습
미ACK 재수신 시뮬레이션
Consumer 2개 분배

질문 · 피드백 환영합니다